

La résolution d'un labyrinthe

Chen Shenglan ; Elamri Imen ; Lahmar Radhia

Mini-projet pour le cours « IHM Interface Robotique »

Année 2025-2026

Titre : La résolution d'un labyrinthe

Résumé :

Ce projet présente une étude comparative de deux stratégies de navigation autonome pour un robot mobile au sein d'un labyrinthe : une approche réactive basée sur la règle de la main droite et une approche planifiée utilisant l'algorithme A* piloté par une machine à états. La méthode réactive s'appuie exclusivement sur les données du capteur LiDAR, segmentées en trois zones (gauche, centre, droite), pour assurer un suivi de mur robuste et déterministe. En parallèle, une solution de navigation globale a été implémentée, combinant une planification de trajectoire sur grille d'occupation avec une gestion précise de l'interaction physique pour la capture d'un objectif (balle).

Le système a été testé dans un environnement de simulation et évalué à l'aide de métriques telles que le temps de navigation et la distance parcourue. Les résultats démontrent que les deux algorithmes permettent une navigation efficace, chacun répondant à des exigences spécifiques : la stabilité déterministe pour la règle de la main droite et la polyvalence fonctionnelle pour l'algorithme A*. Toutefois, certaines limites subsistent. Des améliorations futures pourraient inclure l'intégration de techniques de cartographie de l'environnement (SLAM).

Table des matières

Introduction.....	1
Etat de l'art.....	1
1. Paradigme.....	1
2. Stratégie de planification.....	1
Objectifs.....	4
Méthodes.....	5
Algorithmes.....	5
1. Wall-Following (Main droite) :.....	5
2. Machine à États Finis (FSM) avec A*.....	8
Données.....	10
Métriques.....	11
Résultats.....	12
Discussion.....	14
Limites.....	15
Extensions possibles.....	15
Contributions.....	15
Remerciements.....	16
Bibliographie.....	17
Annexes.....	19
Annexe 1 : Code.....	19

Introduction

Ce projet s'inscrit dans la problématique de la résolution d'un labyrinthe par le robot, consistant à déterminer un itinéraire entre un point de départ et une cible au sein d'un environnement inconnu ou partiellement exploré. L'objectif est de concevoir et d'implémenter un contrôleur réactif permettant au robot mobile de naviguer de manière autonome en s'appuyant exclusivement sur l'analyse en temps réel des données d'un capteur laser. Ce défi technique illustre la complexité de l'intégration capteur-actionneur où la pertinence du système repose sur sa capacité à transformer des données brutes en décisions de navigation fiables. La maîtrise de ces algorithmes est cruciale pour le déploiement de robots dans des missions où l'absence d'intervention humaine exige une autonomie décisionnelle rigoureuse face aux obstacles imprévus.

Etat de l'art

1. Paradigme

La problématique de la navigation robotique demeure, depuis des décennies, un enjeu central pour les chercheurs du domaine. La résolution d'un problème tel que l'exploration d'un labyrinthe impose de définir préalablement l'architecture cognitive du système, c'est-à-dire la manière dont le robot transforme l'information sensorielle en commandes motrices. Historiquement, la littérature a structuré cette réflexion autour de trois paradigmes fondamentaux qui définissent la relation entre la perception, la modélisation et l'action (Antich Tobaruela & Ortiz Rodríguez, 2017).

Le paradigme délibératif, souvent conceptualisé par le modèle « Sense-Plan-Act ». Cette approche repose sur la construction exhaustive d'un modèle de monde global (World Model) à partir de la fusion des données sensorielles. Une fois cette représentation interne établie, le système déploie des algorithmes de recherche sur graphe, tels que A* ou Dijkstra, pour planifier une trajectoire hors-collision optimale. Cependant, la nécessité de maintenir une carte détaillée exige des ressources computationnelles et mémorielles incompatibles avec l'embarqué à bas coût, et toute modification dynamique de l'environnement rend le plan global instantanément obsolète.

En rupture avec cette approche lourde, le paradigme réactif, ou modèle « Sense-Act », a émergé à la fin des années 1980, notamment sous l'impulsion des architectures de subsomption de Rodney Brooks (1991). Ce modèle postule qu'une intelligence de navigation peut émerger sans représentation interne, par le couplage direct et instantané entre les capteurs et les actionneurs. Le comportement global est alors décomposé en modules concurrents (évitement, errance, attraction vers un but) traitant un flux sensoriel local. L'avantage majeur réside dans la réactivité en temps réel face aux obstacles dynamiques. Néanmoins, l'absence de mémoire spatiale et de planification globale engendre une myopie structurelle. Le robot génère fréquemment des trajectoires sous-optimales et demeure extrêmement vulnérable aux minima locaux, tels que les impasses ou les obstacles en « U », menant souvent à l'échec de la mission.

Pour pallier ces déficiences respectives, l'architecture hybride s'est imposée en déléguant la planification globale de la mission (la stratégie) au paradigme délibératif, et l'exécution ainsi que l'évitement d'urgence (la tactique) au paradigme réactif, le robot naviguant de point de passage (waypoint) en point de passage.

2. Stratégie de planification

Les stratégies de planification se divisent fondamentalement en deux types selon la disponibilité de l'information environnementale : la planification hors ligne ou globale (Global/Off-line), où le robot

obtient des informations globales et complètes au préalable sans l'assistance de capteurs, et la planification en ligne ou locale (Local/On-line), où le robot s'appuie sur des capteurs embarqués locaux pour construire une carte en temps réel (Gul et al., 2019).

Pour l'approche globale, la littérature propose de nombreuses approches pour aborder la planification de trajectoire, telles que les champs de potentiels artificiels (APF) (Khatib, 1986), A*, D*, les arbres exploratoires à évolution rapide (RRT) (LaValle, 1998) et les feuilles de route probabilistes (PRMs) (Kavraki et al., 2002). Les avantages des approches globales résident dans le fait que le chemin calculé peut être optimal en termes de distance euclidienne parcourue, et que le robot ne sera piégé dans aucun environnement. Cependant, elles supposent que le robot possède une carte complète du labyrinthe (ou qu'il a construit une carte parfaite lors de la phase d'exploration), et postulent que l'environnement est totalement statique.

L'approche locale est connue sous le nom d'évitement d'obstacles ou de navigation réactive. Le robot détermine sa position et son orientation et contrôle son mouvement via des capteurs externes (tels que des capteurs infrarouges, des capteurs à ultrasons, des capteurs laser et des capteurs visuels). Par exemple, un LiDAR bidimensionnel peut fournir un tableau de distances et d'angles de haute précision de l'environnement immédiat du robot. La tâche centrale de l'algorithme de navigation réactive consiste à transformer en temps réel ce tableau unidimensionnel de distances (ainsi que la position relative de la cible) en commandes de contrôle de mouvement pour le robot. Les principaux avantages des techniques locales résident dans leur faible complexité algorithmique et leur réponse rapide. Les approches locales peuvent efficacement éviter les collisions avec les obstacles.

a. L'approche locale

Les méthodes fondamentales basées sur les capteurs s'appuient généralement de manière directe sur les données sensorielles pour effectuer une cartographie locale, un ajustement de direction ou un évitement d'obstacles. Le LiDAR est capable de fonctionner de manière autonome pour cartographier l'environnement. Lorsqu'il est combiné avec une caméra ou un système de navigation inertielle, il peut servir de puissant outil pour aider à corriger automatiquement et à distance sa position et son orientation.

La méthode des champs de potentiels artificiels est une approche classique en navigation réactive locale. Le robot se déplace dans un espace constitué d'un « champ de forces », passant d'un état de haute énergie vers un état de basse énergie. Lors de la navigation, le point cible est défini comme un « pôle attractif » qui attire le robot vers lui, tandis que les obstacles sont définis comme des « surfaces répulsives » qui repoussent le robot pour éviter les collisions. En mettant rapidement à jour la matrice du champ de forces de l'environnement, le robot peut efficacement éviter les obstacles dynamiques. Les problèmes fréquemment rencontrés avec cette navigation réactive locale sont, par exemple, le problème des minima locaux et le problème du canyon, c'est-à-dire que le robot peut se retrouver bloqué là où la force de répulsion des obstacles et la force d'attraction de la cible s'annulent.

Afin de gérer la complexité de l'environnement et l'imprécision des données sensorielles, la navigation locale utilise largement les techniques de calcul souple issues de l'intelligence artificielle, incluant principalement les algorithmes déterministes, les algorithmes non déterministes/stochastiques ainsi que les algorithmes évolutionnaires/hybrides.

Les algorithmes déterministes englobent principalement la logique floue (Fuzzy Logic), adaptée au traitement des informations imprécises issues des mesures des capteurs et aux connaissances basées sur

des heuristiques (Montaner & Ramirez-Serrano, 1998), ainsi que les réseaux de neurones (Neural Network, NN), inspirés du système nerveux biologique (la façon dont le cerveau traite l'information) et composés de neurones artificiels interconnectés (Engedy & Horvath, 2010).

Les algorithmes non déterministes sont pour la plupart des algorithmes d'intelligence en essaim bio-inspirés, utilisant une fonction d'aptitude (fitness) pour rechercher le chemin optimal dans des environnements complexes. Cela inclut les algorithmes génétiques, qui s'inspirent de l'évolution biologique et de la sélection naturelle (Ghorbani, Shiry, & Nodehi, 2009). À partir d'une population initiale, et par l'évaluation de l'aptitude, ils utilisent des opérations de « sélection », « croisement » et « mutation » pour générer continuellement la génération suivante, jusqu'à ce que la population converge vers la solution de trajectoire optimale. Cela comprend également l'optimisation par essaim particulaire (Particle Swarm Optimization, PSO), où chaque « particule » du système représente une solution potentielle. Les particules ajustent continuellement leur vitesse de vol et leur position en fonction de leur propre meilleure position historique (Pbest) et de la meilleure position de l'ensemble de l'essaim (Gbest), convergeant ainsi rapidement vers le meilleur chemin (Ahmadzadeh & Ghanavati, 2012).

Un algorithme unique peut rarement satisfaire simultanément toutes les exigences telles que la robustesse, la vitesse de réponse et la précision ; la navigation locale adopte donc souvent des algorithmes hybrides. C'est le cas, par exemple, des systèmes neuro-flous (Neuro-Fuzzy), qui combinent la capacité d'apprentissage et de généralisation des réseaux de neurones avec la capacité de la logique floue à traiter l'incertitude (Zhu & Yang, 2007). Généralement, les réseaux de neurones sont utilisés pour ajuster les paramètres des fonctions d'appartenance dans le système flou, améliorant ainsi considérablement les performances de navigation et d'évitement d'obstacles dans des environnements inconnus.

b. L'approche globale

Selon Zhang et al. (2018), l'approche globale pour les robots mobiles autonomes repose sur trois principes fondamentaux : la modélisation de l'environnement, la définition des critères d'optimisation et le déploiement d'un algorithme de recherche.

Le processus de la modélisation de l'environnement intervient impérativement en amont du mouvement, durant une phase dite « hors ligne » (offline), où le robot appréhende la topologie de son espace de travail. Cette étape initiale consiste à identifier la localisation des obstacles ainsi que les positions source et cible, permettant ainsi de transcrire la réalité physique en une carte environnementale exploitable. Pour structurer cette représentation, plusieurs paradigmes de modélisation coexistent. La décomposition en cellules (Cell Decomposition) consiste à segmenter l'espace continu en un ensemble de domaines discrets et non chevauchants, simplifiant ainsi la complexité géométrique en un modèle mathématique rigoureux (Prasad et al., 2024). Cette approche se décline généralement sous des formes exactes, approximatives ou probabilistes (Atyabi & Powers, 2013 ; Sánchez-Ibáñez et al., 2021). Parallèlement, l'approche par feuilles de route (Roadmap Approach) privilégie la génération d'un réseau de connectivité unidimensionnel, tel que les graphes de visibilité ou les diagrammes de Voronoï, pour condenser l'espace libre en un graphe de recherche (Campbell et al., 2020). À l'opposé de ces représentations discrètes, la méthode des champs de potentiel artificiel (Artificial Potential Field) introduit une dimension continue en modélisant l'environnement via des forces énergétiques virtuelles, où l'attraction de l'objectif et la répulsion des obstacles guident intrinsèquement le système (Bayat et al., 2018 ; Lin et al., 2022).

Une fois l'environnement modélisé, la détermination de la trajectoire optimale nécessite l'établissement de critères de sélection précis. Ces standards permettent d'évaluer la pertinence d'un chemin selon des métriques de distance, de temps ou d'efficacité énergétique. Un planificateur robuste doit opérer un arbitrage complexe entre des objectifs souvent divergents, tels que la sécurité du parcours, la fluidité cinématique et la minimisation de la consommation de ressources. Dans le cadre des approches heuristiques, ces exigences sont formalisées par une fonction de coût. À titre d'exemple, l'algorithme A* intègre ces critères à travers sa fonction d'évaluation $f_n = g_n + h_n$, où le coût accumulé depuis l'origine et l'estimation heuristique du reste à parcourir h_n sont considérés, visant ainsi une efficacité de recherche maximale sans sacrifier l'optimalité.

Le succès opérationnel de la planification dépend in fine de la performance de l'algorithme de recherche de chemin. Si les méthodes conventionnelles issues de la décomposition ou des champs de potentiel sont efficaces dans des cadres statiques et parfaitement connus, elles manifestent des limites computationnelles ou des risques de stagnation dans des minima locaux face à des environnements complexes. Les algorithmes de recherche modernes, tels que Dijkstra, pallient ces lacunes en explorant systématiquement les nœuds du graphe par ordre de coût cumulé (Tang et al., 2025), bien que cette exhaustivité puisse s'avérer coûteuse en temps de calcul (Gonzalez et al., 2017). Toutefois, contrairement à cette approche exhaustive qui vise à découvrir toutes les solutions potentielles au prix d'un coût computationnel élevé, l'algorithme A* privilégie l'identification directe de la solution de coût minimal. En orientant la recherche vers la cible, A* évalue chaque nœud adjacent en combinant le coût déjà accumulé et une estimation du coût restant pour atteindre l'objectif. Outre ces standards, la littérature propose des extensions spécialisées telles que D*, Theta*, ou les arbres aléatoires à exploration rapide (RRT), ainsi que des méthodes de recherche classiques (BFS, DFS, UCS). Enfin, des techniques fondées sur l'échantillonnage, à l'instar des Fast Marching Trees (FMT), complètent ce panorama en offrant des solutions adaptées aux espaces de haute dimensionnalité (Agrawal et al., 2022).

Objectifs

L'objectif principal de ce projet est de concevoir et d'évaluer un système de navigation autonome permettant à un robot mobile d'évoluer dans un environnement de type labyrinthe en utilisant uniquement les informations issues de ses capteurs. Plus précisément, ce projet poursuit les objectifs suivants :

- **Implémentation d'une navigation basée sur capteurs:** l'objectif principal consiste à transformer les données du capteur laser (LiDAR) en commandes de mouvement pour le robot. Le robot doit analyser les distances aux obstacles dans différentes directions et adapter dynamiquement sa trajectoire afin d'éviter les collisions et de progresser dans le labyrinthe.
- **Évaluation des performances du système via des métriques quantitatives:** Une fois la méthode implémentée, on évalue son efficacité dans l'environnement de simulation en se basant sur des indicateurs mesurables tels que le temps pour atteindre la sortie du labyrinthe, la distance totale parcourue par le robot et la capacité du robot à éviter les obstacles. Ces métriques permettent d'analyser les performances de l'algorithme et d'identifier ses points forts ainsi que ses limites.

Méthodes

La résolution du problème de navigation dans le labyrinthe est abordée à travers l'implémentation de deux algorithmes distincts et indépendants. Cette double approche permet d'évaluer et de confronter une stratégie d'exploration purement réactive à une stratégie de planification globale.

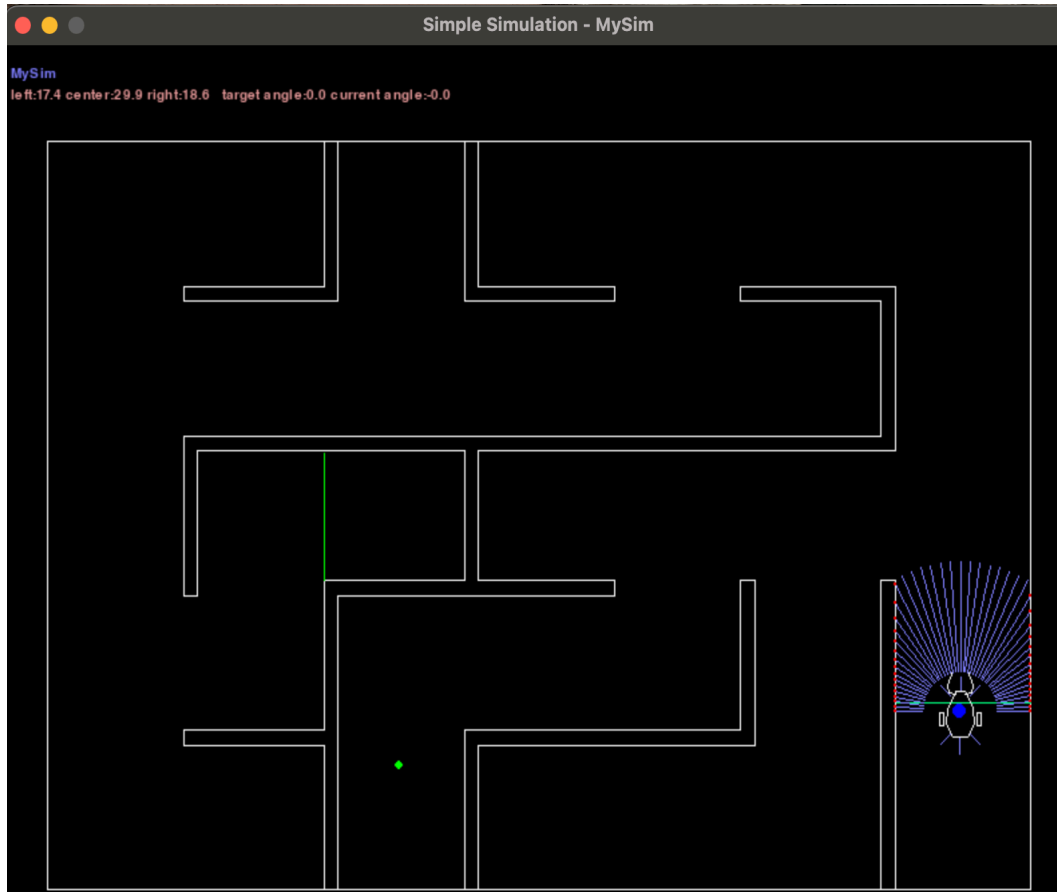


Figure 1 : Environnement de simulation du labyrinthe.

Algorithmes

1. Wall-Following (Main droite) :

La première méthode proposée repose sur une navigation réactive basée exclusivement sur les données du laser scan. Le robot divise son champ de perception en trois zones (gauche, centre, droite) et adapte son comportement en fonction des distances détectées. Le principe général suit une règle de navigation dite right-hand rule (suivi de mur à droite), permettant au robot d'explorer le labyrinthe tout en évitant les collisions. L'orientation cible du robot est ajustée dynamiquement selon la configuration observée.

- **Extrait de code Python:**

L'algorithme s'articule autour de quatre parties importantes : la perception, la décision, le contrôle du mouvement et l'action.

La perception : Ce bloc segmente les données de laser brute en 3 secteurs (droite, centre, gauche) et les compresse en 3 scalaires via une moyenne pour alimenter la logique de décision. Cette méthode est computationnellement efficace. Mais cette réduction de dimensionnalité entraîne une perte d'information, le système sacrifie la topologie spatiale locale, le limitant à une perception macroscopique.

```
laser_data = np.array(self.laserscan.values)

n = len(laser_data)

k = n//3

right_data = laser_data[:k]

center_data = laser_data[k:2*k]

left_data = laser_data[2*k:]

right = right_data.mean()

center = center_data.mean()

left = left_data.mean()
```

Le logique de la décision : Lorsque les distances mesurées dans les trois directions sont simultanément inférieures à 18,0, l'algorithme identifie une impasse et inverse directement l'angle cible de 180 degrés. Par ailleurs, si la distance frontale est inférieure à 18,0, le robot ordonne une rotation de 90 degrés vers la gauche afin de conserver le mur sur son droit. Enfin, lorsque la valeur minimale du côté droit excède 27,0 et que cette condition se maintient au-delà de 1,2 seconde, le robot ordonne une rotation de 90 degrés vers la droite.

```
if max(right, center, left) < 18.:

    self.target_angle = normalize_angle(self.target_angle + np.pi)

elif center < 18.:

    self.target_angle = normalize_angle(self.target_angle + np.pi/2)

elif right_data.min() > 27.:

    if self.time_right_is_empty < 0:

        self.time_right_is_empty = fw.time

    if self.time_right_is_empty > 0 and fw.time >= self.time_right_is_empty + 1.2:

        self.target_angle = normalize_angle(self.target_angle - np.pi/2)
```

```
self.time_right_is_empty = -1
```

Le contrôle du mouvement : Ce segment de code implémente une loi de commande non linéaire basée sur une décroissance exponentielle : $v_{angular} = s \cdot (1 - e^{-\frac{5|\Delta\theta|}{\pi}})$. Lorsque l'erreur angulaire $|\Delta\theta|$ est importante, le multiplicateur tend vers 1, ce qui génère une sortie proche de la vitesse angulaire maximale. À l'inverse, lorsque l'erreur tend vers 0, la commande subit une décroissance exponentielle et fluide. Cette stratégie inhibe efficacement les dépassements de consigne (overshoot) et les oscillations du système

```
def speed_multiplier(current, target):

    delta_angle = normalize_angle(target - current)

    s =sgn(delta_angle)

    return s*(1-np.exp(-5*abs(delta_angle)/math.pi))
```

Ce segment de code définit une marge de tolérance de 5e-2 degrés. Lorsque l'erreur angulaire excède cette valeur, le robot exécute une rotation pure sur place. Lorsque l'erreur devient inférieure à ce seuil, le système bascule dans un état de translation rectiligne couplé à de micro-corrrections angulaires

```
if abs(delta_angle) > 5e-2:

    motor_control = (0., self.turn_speed*speed_multiplier(current_angle,
self.target_angle))

else:

    motor_control = (self.forward_speed, sgn(delta_angle)*5e-2)
```

L'action : Pour réaliser les contrôles , on utilise ce segment de code afin de matérialiser la logique de contrôle. Plus précisément, les commandes abstraites de vitesse linéaire (v) et angulaire (ω) issues de la logique de contrôle sont converties en consignes spécifiques pour chaque roue via les équations $v_{left} = v - \frac{\omega}{2}$ et $v_{right} = v + \frac{\omega}{2}$. Ces valeurs sont ensuite directement transmises aux actionneurs des pneumatiques (update_drive).

```
v_l = motor_control[0] - motor_control[1]/2

v_r = motor_control[0] + motor_control[1]/2

self.tires[0].update_drive( v_l ) # left wheel

self.tires[1].update_drive( v_r ) # right wheel
```

2. Machine à États Finis (FSM) avec A*

La navigation globale du robot est piloté par une Machine à États Finis (FSM) combinée à un algorithme de planification de chemin A* et un évitement d'obstacles réactif par laser. Cette approche se déroule en trois étapes principales. Tout d'abord, la cartographie consiste à représenter l'environnement sous forme d'une grille 2D, en tenant compte du rayon du robot pour éviter les obstacles. Ensuite, la recherche de trajectoire utilise l'algorithme A* pour déterminer le chemin le plus court entre la position actuelle du robot et la sortie du labyrinthe. Enfin, le lissage de la trajectoire supprime les points superflus afin de créer un chemin fluide et directement exécutable par les contrôleurs du robot.

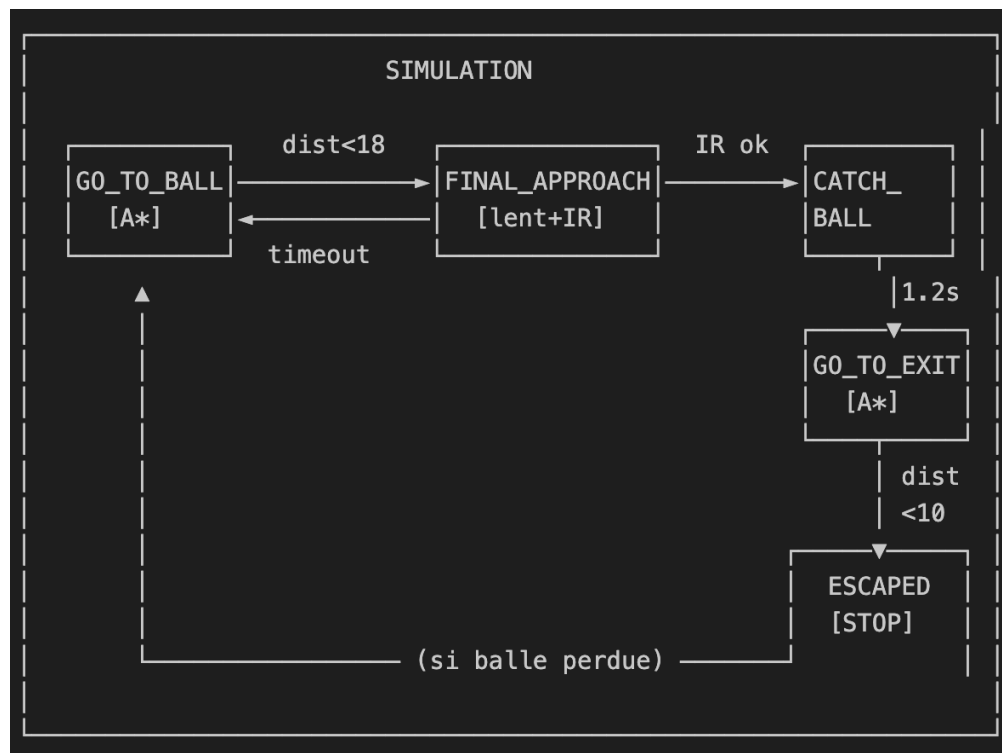


Figure 2 : Schéma récapitulatif.

- **Extrait de code Python:**

Dans notre système, la planification globale du chemin est réalisée à l'aide de l'algorithme A* appliqué sur une grille d'occupation représentant l'environnement du labyrinthe. Cette méthode permet au robot de calculer un chemin optimal entre sa position actuelle et l'objectif tout en évitant les obstacles.

Initialisation et structures de données:

La fonction h représente l'heuristique utilisée par A*, qui correspond ici à la distance euclidienne entre un nœud et la position cible.

```

open_heap = []

    heapq.heappush(open_heap, (0.0, start))

    came_from = {}

    gscore = {start: 0.0}

    def h(a, b):

        return math.hypot(a[0] - b[0], a[1] - b[1])

```

Exploration des cellules voisines:

Dans cette étape, l'algorithme explore les cellules voisines du nœud courant dans la grille. Les positions situées en dehors de la grille ou correspondant à des obstacles sont ignorées. Pour chaque cellule accessible, l'algorithme calcule un coût temporaire correspondant à la distance parcourue depuis la position de départ. Si ce coût est plus faible que celui déjà enregistré, la cellule est mise à jour et ajoutée dans la file de priorité afin d'être explorée ultérieurement.

```

for dx, dy, cost in neighbors:

    nx = current[0] + dx

    ny = current[1] + dy

    if not (0 <= nx < self.occ.shape[1] and 0 <= ny <
self.occ.shape[0]):

        continue

    if self.occ[ny, nx] != 0:

        continue

    tentative = gscore[current] + cost

    nxt = (nx, ny)

    if tentative < gscore.get(nxt, 1e18):

        came_from[nxt] = current

        gscore[nxt] = tentative

```

```
f = tentative + h(nxt, goal)

heapq.heappush(open_heap, (f, nxt))
```

Reconstruction du chemin optimal :

Lorsque l'objectif est atteint, l'algorithme reconstruit le chemin optimal en remontant les parents enregistrés dans *came_from*. Le chemin est ensuite inversé afin d'obtenir la trajectoire depuis la position de départ jusqu'à la cible.

```
if current == goal:

    path = [current]

    while current in came_from:

        current = came_from[current]

        path.append(current)

    path.reverse()

    return [self._grid_to_world(px, py) for px, py in path]
```

L'utilisation du A*:

l'algorithme A* est utilisé pour calculer un chemin entre la position actuelle du robot et l'objectif souhaité.

```
def _plan_to(self, goal_xy):

    pos = self.b2body.position

    raw = self._astar((pos[0], pos[1]), goal_xy)

    if not raw:

        self.path_points = []

        self.path_index = 0

        return

    simp = self._simplify_path(raw)

    self.path_points = simp

    self.path_index = 0
```

Données

Dans ce projet, aucune base de données externe n'est utilisée. Les données exploitées proviennent exclusivement du capteur laser (LiDAR) embarqué sur le robot simulé. Ce capteur fournit en continu des mesures de distance entre le robot et les obstacles présents dans son environnement immédiat. Les informations collectées correspondent à un ensemble de distances mesurées dans différentes directions, couvrant un demi-cercle frontal autour du robot.

Chaque mesure représente la distance entre le robot et l'obstacle le plus proche dans une direction donnée. Ces données permettent au robot d'obtenir une perception locale de son environnement et des obstacles qui peuvent empêcher le robot d'atteindre la sortie .

Afin de simplifier le processus de prise de décision, les données issues du laser scan sont segmentées en trois zones principales correspondant aux secteurs gauche, centre et droite du champ de perception. Pour chaque secteur, les distances mesurées sont analysées afin d'estimer la proximité des obstacles. Cette segmentation permet de faciliter l'application des règles de navigation, telles que l'évitement d'obstacles ou le suivi de mur.

Ces informations sont ensuite utilisées en temps réel par l'algorithme de navigation pour déterminer les actions à effectuer par le robot, notamment avancer, tourner ou effectuer un demi-tour lorsque des obstacles sont détectés à proximité.

Métriques

Plusieurs métriques quantitatives ont été définies pour évaluer la performance du système de navigation. Ces indicateurs permettent d'analyser l'efficacité du robot lors de son déplacement dans le labyrinthe et d'évaluer la qualité de l'algorithme de navigation.

1. Temps de navigation :

Le temps de navigation correspond à la durée totale (en secondes) nécessaire au robot pour atteindre la sortie du labyrinthe à partir de sa position initiale. Cette métrique permet d'évaluer l'efficacité temporelle de l'algorithme de navigation : un temps plus court indique généralement que le robot parvient à trouver la sortie plus rapidement.

$$T_{navigation} = t_{sortie} - t_{départ}$$

Le $t_{départ}$ représente le moment de début de la simulation et t_{sortie} le moment où le robot atteint la sortie du labyrinthe.

Dans l'implémentation du système, ces valeurs sont enregistrées directement pendant la simulation grâce aux variables *start_time* et *finish_time*, permettant de calculer automatiquement la durée totale de la mission.

2. Distance parcourue :

Elle correspond à la distance totale (en unités du monde simulé) parcourue par le robot entre le point de départ et la sortie. Cette métrique permet d'évaluer l'efficacité du trajet suivi. Un trajet plus court indique une navigation plus efficace et plus proche d'un chemin optimal.

La distance est calculée en accumulant les déplacements successifs du robot entre deux instants de simulation :

$$D = \sum \sqrt{(x_i - x(i-1))^2 + (y_i - y(i-1))^2}$$

où (x_i, y_i) représente la position du robot à l'instant i .

3. Capacité d'évitement des obstacles:

Cette métrique permet d'évaluer la robustesse de l'algorithme face aux obstacles dans le labyrinthe. Elle peut être analysée en vérifiant si le robot est capable de naviguer dans le labyrinthe sans entrer en collision avec les murs. Un bon système de navigation doit permettre au robot d'éviter les obstacles tout en continuant à progresser dans l'environnement.

Un compteur est incrémenté chaque fois que la distance minimale mesurée devant le robot est inférieure à un certain seuil fixé à distance < 8 dans le système implémenté.

Chaque fois que cette condition est satisfaite, le compteur *near_obstacle_count* est augmenté.

4. Taux de réussite de la mission :

Le taux de réussite indique le pourcentage de simulations dans lesquelles le robot parvient à atteindre la sortie du labyrinthe. Cette métrique permet d'évaluer l'efficacité globale du système de navigation.

$$T_{succès} = N_{succès} / N_{tests} \times 100$$

où :

- *Nsuccès* correspond au nombre de simulations réussies : le robot atteint la sortie du labyrinthe.
- *Ntests* correspond au nombre total de simulations réalisées.

Dans le cadre de ce projet, la fonctionnalité optionnelle de capture de balle avec l'algorithme de la main droite n'a pas été implémentée.

Ces indicateurs métriques permettent d'analyser les performances de l'algorithme et d'identifier ses points forts ainsi que ses limites dans l'environnement simulé: l'efficacité temporelle, la qualité du chemin suivi, la sécurité de la navigation et la robustesse globale du système.

Résultats

Afin d'évaluer les performances du système de navigation implémenté, on a réalisé plusieurs simulations dans l'environnement de test contenant le labyrinthe. Le robot est initialisé à une position fixe et doit atteindre la sortie en utilisant uniquement les données du capteur LiDAR et l'algorithme de navigation basé sur la règle de la main droite.

Lors des différents tests, le robot est capable de se déplacer dans le labyrinthe tout en évitant les collisions avec les murs. La méthode utilisée aide le robot à analyser en continu les distances mesurées par le capteur laser et d'adapter sa direction dans le trajet.

Les métriques définies précédemment ont été utilisées pour analyser le comportement du robot pendant les essais. Le tableau présente un exemple de résultats obtenus lors de plusieurs tests.

Simulation	Temps de navigations (s)	Distance parcourue (unité)	Succès
Test 1	264,5	820	atteindre la sortie sans capturer la balle
Test 2	265	820	atteindre la sortie sans capturer la balle
Test 3	264,2	820	atteindre la sortie sans capturer la balle

Tableau 1 : Résultats obtenus lors de simulation (main droite)

Simulation	Temps de navigations (s)	Distance parcourue (unité)	Succès
Test 1	390,7	992	atteindre la sortie avec la balle
Test 2	463,7	1091	atteindre la sortie avec la balle
Test 3	318,2	841	atteindre la sortie avec la balle

Tableau 2 : Résultats obtenus lors de simulation (A*)

Les résultats montrent que le robot parvient à atteindre la sortie du labyrinthe dans l'ensemble des simulations réalisées. Le temps nécessaire pour trouver la sortie varie en fonction de la trajectoire suivie par le robot lors de l'exploration du labyrinthe.

Les résultats du Tableau 1 démontrent une consistance parfaite de la règle de la main droite avec une distance parcourue identique de 820u sur l'ensemble des essais et un temps de navigation moyen de

264,6s avec un écart maximal de 0,4s. Cet algorithme prouve que sa séquence d'exécution est strictement liée aux entrées sensorielles locales et à la topologie du milieu. Cette stabilité exceptionnelle assure une navigation prévisible et robuste dans le labyrinthe.

À l'invers, l'algorithme A* présente une variabilité plus marquée selon les résultats du Tableau 2. La fluctuations de distance atteignant 250 u et l'écart du temps maximal est de 145,5s. Les délais observés dans les tests 1 et 2 résultent principalement de la machine à états lors de l'approche finale de la balle, où la précision requise pour la capture peut entraîner des manœuvres de récupération répétées. Cependant cette approche a sa polyvalence fonctionnelle. Le test optimal (Test 3), avec seulement 21 unités de distance supplémentaires par rapport à la main droite, parvient à accomplir une mission complète incluant la capture et la sortie

Globalement, les essais montrent que l'algorithme permet une navigation stable et robuste dans l'environnement simulé. Le robot parvient toujours à éviter les obstacles et à progresser dans le labyrinthe jusqu'à atteindre la sortie, ce qui valide les objectifs principaux du projet.

Discussion

Dans ce projet, nous avons implémenté et testé deux méthodes distinctes pour la navigation autonome dans un labyrinthe : l'algorithme réactif basé sur la règle de la main droite et l'approche globale piloté par une Machine à États Finis (FSM) combiné à l'algorithme A*.

Les simulations réalisées pour les deux approches de navigation fournissent un éclairage précis sur leurs performances respectives.

- **Navigation réactive (Main droite) :**

Les résultats obtenus montrent que la stratégie de navigation réactive implémentée est capable de résoudre le problème de sortie du labyrinthe dans un environnement inconnu. L'utilisation des données du capteur LiDAR permet au robot d'adapter son comportement en temps réel en fonction de la configuration locale de l'environnement. En suivant systématiquement le mur situé à droite, le robot est capable de progresser dans l'environnement sans nécessiter de carte globale ni de planification complexe.

Cette méthode présente plusieurs avantages : elle est computationnellement légère, ce qui la rend adaptée à des systèmes embarqués disposant de ressources limitées et elle permet une réaction immédiate face aux obstacles, ce qui garantit la sécurité de la navigation. Cependant, cette approche présente aussi certaines limites. En l'absence de représentation interne de l'environnement, le robot ne peut trouver son trajet ni anticiper les obstacles ou impasses. Il peut aussi utiliser des trajectoires longues et inefficaces avant de trouver la sortie. De plus, la performance du système dépend fortement du choix des seuils utilisés pour détecter les obstacles. Un mauvais réglage de ces paramètres peut entraîner des comportements instables ou des collisions.

- **Navigation globale (FSM+A*) :**

L'approche globale combine la planification de chemin A* avec une FSM et un évitement d'obstacles réactifs. Elle se base sur la construction préalable d'une carte de l'environnement, la recherche du chemin le plus court et le lissage de la trajectoire. Cette technique aide le robot à suivre un itinéraire optimal, de réduire la distance parcourue et de limiter le temps nécessaire pour atteindre la sortie.

- **Comparaisons**

D'une part, la règle de la main droite se distingue par une stabilité absolue, tant au niveau du temps que de la distance parcourue. Cette performance s'explique par une logique de décision simplifiée qui élimine toute incertitude. Cependant, cette robustesse est limitée vis-à-vis des objectifs secondaires. D'autre part, l'algorithme A* fait preuve d'une flexibilité supérieure. Bien que ses métriques soient plus fluctuantes, il parvient à résoudre la tâche de manière plus complète. Les variations observées dans le Tableau 2 doivent être interprétées comme le coût de l'incertitude liée à l'interaction physique. Les manœuvres de récupération répétées lors de la capture de la balle (phases de stuck/recovery) augmentent le temps et la distance, mais elles témoignent de la capacité du système à corriger ses erreurs pour atteindre son but.

Dans ce contexte, l'efficacité ne peut être définie simplement par la minimisation du temps ou de la distance. Si l'on définit l'efficacité comme la valeur de la tâche accomplie par unité de temps, l'A* l'emporte sur la main droite. Le Test 3 en est la preuve irréfutable : il démontre qu'en conditions optimales, l'A* peut accomplir une mission complexe (capture + sortie) avec un surcoût de distance marginal (seulement 21u de plus que la main droite). Cela prouve que l'intelligence algorithmique permet de maximiser le succès de la mission au prix d'un investissement minimal en ressources supplémentaires.

Limites

Malgré les résultats satisfaisants obtenus lors des simulations, les deux approches méthode proposées présentent plusieurs limites.

Concernant la méthode de suivi de mur, tout d'abord, la technique de suivi de mur peut conduire le robot à effectuer des boucles dans certaines labyrinthes complexes, en particulier lorsque la structure de l'environnement crée des circuits fermés. De plus, la trajectoire suivie par le robot n'est généralement pas optimale, car la stratégie de suivi de mur ne garantit pas le chemin le plus court vers la sortie. Le robot peut donc parcourir une distance plus longue que la distance optimale. Par ailleurs, on note que le système ne possède aucune mémoire spatiale: il ne conserve aucune information sur les zones déjà explorées et peut ainsi repasser plusieurs fois au même endroit. Le comportement du robot dépend aussi de multiples seuils de décisions basés liés aux distances mesurées par le capteur LiDAR; ces paramètres doivent être ajustés empiriquement et peuvent varier selon la configuration de l'environnement. Enfin, la méthode actuelle ne prend pas en compte certains cas particuliers, comme l'absence d'obstacles isolés à l'intérieur du labyrinthe (îlots), ce qui peut perturber la navigation et compliquer l'exploration de l'espace.

L'algorithme A* présente des limites liées à sa complexité computationnelle et à sa dépendance envers une carte statique. Contrairement à la méthode réactive, il nécessite une connaissance préalable de l'environnement pour construire sa grille d'occupation. De plus, les résultats du Tableau 2 soulignent une instabilité lors des phases d'interaction physique : l'écart entre le planificateur global et l'exécution locale peut engendrer des cycles de recalcul fréquents (stuck/recovery), rendant le temps d'exécution sensible aux erreurs de précision lors de la capture de l'objet.

Extensions possibles

- **Cartographie de l'environnement (SLAM)** : l'intégration d'une méthode de cartographie simultanée et de localisation (SLAM) permettrait au robot de construire progressivement une représentation du labyrinthe. Cette carte pourrait ensuite être utilisée pour planifier un chemin optimal vers la sortie.

- **Amélioration de la perception** : Au lieu de réduire les données du laser scan à trois valeurs moyennes, une analyse plus fine de la distribution des distances pourrait être réalisée afin d'obtenir une perception plus précise de l'environnement.

Contributions

Ce projet a été réalisé en collaboration, l'organisation du travail s'est faite en deux étapes. Dans un premier temps, chaque membre du groupe a travaillé individuellement sur certaines parties du projet ou sur des fonctionnalités spécifiques du système. Dans un second temps, des séances de travail à distance ont été organisées via Google Meet, durant lesquelles nous avons effectué des revues collectives, discuté et testé des choix techniques et validé les différentes contributions.

Par ailleurs, deux amis nous ont apporté de l'aide en nous donnant des conseils techniques et en nous aidant à résoudre certains détails d'implémentation. Nous les remercions pour leur soutien lors de certaines phases du projet.

Remerciements

Nous remercions notre professeur Monsieur Salvatore Anzalone pour les cours dispensés ainsi que pour le matériel pédagogique mis à disposition, qui ont permis la réalisation de ce projet.

Bibliographie

Atyabi, A., & Powers, D. (2013). Review of classical and heuristic-based navigation and path planning approaches. *International Journal of Advancements in Computing Technology (IJACT)*, 5(14).

Antich Tobaruela, J., & Ortiz Rodriguez, A. (2017). Reactive navigation in extremely dense and highly intricate environments. *PloS one*, 12(12), e0189008.

Agrawal, R., Singh, B., & Kumar, R. (2022, November). Classical approaches for mobile robot path planning: A review. In *2022 International Conference on Computing, Communication, and Intelligent Systems (ICCCIS)* (pp. 400-405). IEEE.

Ahmadzadeh, S., & Ghanavati, M. (2012). Navigation of mobile robot using the PSO particle swarm optimization. *Journal of Academic and Applied Studies (JAAS)*, 2(1), 32-38.

Brooks, R. A. (1991). Intelligence without representation. *Artificial intelligence*, 47(1-3), 139-159.

Bayat, F., Najafinia, S., & Aliyari, M. (2018). Mobile robots path planning: Electrostatic potential field approach. *Expert Systems with Applications*, 100, 68-78.

Campbell, S., O'Mahony, N., Carvalho, A., Krpalkova, L., Riordan, D., & Walsh, J. (2020, February). Path planning techniques for mobile robots a review. In *2020 6th International Conference on Mechatronics and Robotics Engineering (ICMRE)* (pp. 12-16). IEEE.

Engedy, I., & Horváth, G. (2010, November). Artificial neural network based local motion planning of a wheeled mobile robot. In *2010 11th International Symposium on Computational Intelligence and Informatics (CINTI)* (pp. 213-218). IEEE.

Galceran, E., & Carreras, M. (2013). A survey on coverage path planning for robotics. *Robotics and Autonomous systems*, 61(12), 1258-1276.

Ghorbani, A., Shiry, S., & Nodehi, A. (2009, April). Using genetic algorithm for a mobile robot path planning. In *2009 International Conference on Future Computer and Communication* (pp. 164-166). IEEE.

Gul, F., Rahiman, W., & Nazli Alhady, S. S. (2019). A comprehensive study for robot navigation techniques. *Cogent Engineering*, 6(1). <https://doi.org/10.1080/23311916.2019.1632046>

Gonzalez, R., Kloetzer, M., & Mahulea, C. (2017, October). Comparative study of trajectories resulted from cell decomposition path planning approaches. In *2017 21st International Conference on System Theory, Control and Computing (ICSTCC)* (pp. 49-54). IEEE.

Kavraki, L. E., Svestka, P., Latombe, J. C., & Overmars, M. H. (2002). Probabilistic roadmaps for path planning in high-dimensional configuration spaces. *IEEE transactions on Robotics and Automation*, 12(4), 566-580.doi: 10.1109/70.508439.

Khatib, O. (1986). Real-time obstacle avoidance for manipulators and mobile robots. *Autonomous Robot Vehicles*, Springer. pp. 396–404.

LaValle, S. (1998). Rapidly-exploring random trees: A new tool for path planning. *Research Report 9811*.

Lin, S., Liu, A., Wang, J., & Kong, X. (2022). A review of path-planning approaches for multiple mobile robots. *Machines*, 10(9), 773.

Randria, I., Khelifa, M. M. B., Bouchouicha, M., & Abellard, P. (2007, November). A comparative study of six basic approaches for path planning towards an autonomous navigation. In *IECON 2007-33rd Annual Conference of the IEEE Industrial Electronics Society* (pp. 2730-2735). IEEE.

Montaner, M. B., & Ramirez-Serrano, A. (1998). Fuzzy knowledge-based controller design for autonomous robot navigation. *Expert Systems with Applications*, 14(1-2), 179-186. doi:10.1016/S0957-4174(97)00059-6

Prasad, A., Chaudhary, K., Sharma, B., & Ali, A. S. (2024). Robot path planning and motion control: a systematic review. *Engineered Science*, 33, 1354.

Tang, Y., Zakaria, M. A., & Younas, M. (2025). Path planning trends for autonomous mobile robot navigation: A review. *Sensors*, 25(4), 1206.

Zhu, A., & Yang, S. X. (2007). Neurofuzzy-based approach to mobile robot navigation in unknown environments. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, 37(4), 610-621.

Zhang, H. Y., Lin, W. M., & Chen, A. X. (2018). Path planning for the mobile robot: A review. *Symmetry*, 10(10), 450.

Zafar, M. N., & Mohanta, J. C. (2018). Methodology for path planning and optimization of mobile robots: A review. *Procedia computer science*, 133, 141-152.

Annexes

Annexe 1 : Code

[main_maze \(main droite\).py](#)

[main_maze \(A*\).py](#)